

Lab: Building ASP.NET Web API

with Dapper Micro ORM

Software Engineering

1. Mục tiêu (Lab Objectives)

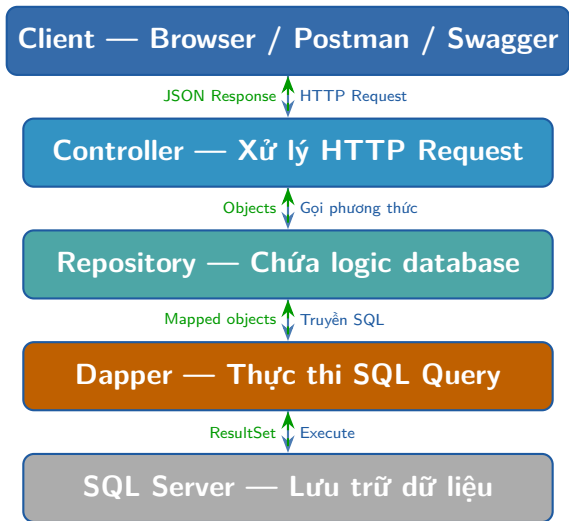
Sau khi hoàn thành lab, sinh viên có khả năng:

- Hiểu cách Dapper kết nối với ASP.NET Core Web API
- Cài đặt CRUD operations sử dụng SQL query thuần
- Áp dụng mẫu thiết kế **Repository Pattern**
- Viết JOIN queries và ánh xạ kết quả với Dapper
- Hiểu luồng request/response của Web API

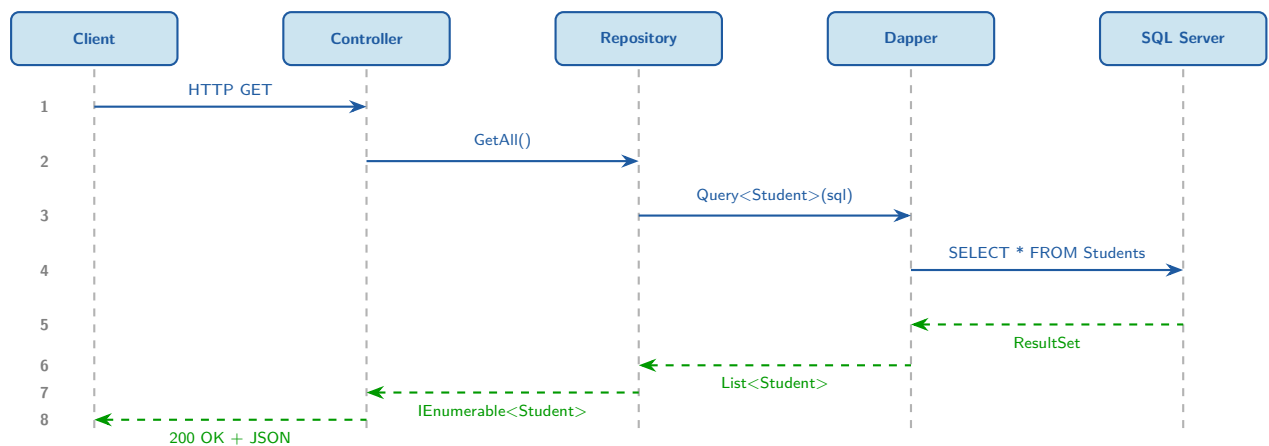
Công nghệ	Phiên bản	Vai trò
ASP.NET Core Web API	.NET 8+	Backend framework
Dapper Micro ORM	2.x	Database access layer
Microsoft SQL Server	2019+	Database

2. Kiến trúc hệ thống (System Architecture)

2.1. Sơ đồ phân lớp (Layered Architecture)



2.2. Sequence Diagram — Luồng GET /api/students



3. Yêu cầu cài đặt (Prerequisites)

- .NET SDK 8+ — <https://dotnet.microsoft.com/download>
- SQL Server — Express hoặc Developer edition
- SQL Server Management Studio (SSMS) hoặc Azure Data Studio
- NuGet packages: Dapper, Microsoft.Data.SqlClient

4. Hướng dẫn thực hành (Step-by-Step)

[x] Bước 1 — Tạo Database và Table

Mở SSMS, kết nối đến SQL Server, rồi chạy script sau:

```
1 CREATE DATABASE SchoolDB;
2 GO
3
4 USE SchoolDB;
5
6 CREATE TABLE Students
7 (
8     Id INT PRIMARY KEY IDENTITY,
9     Name NVARCHAR(100) NOT NULL,
10    Age INT
11 );
12
13 -- Thêm dữ liệu mẫu để test
14 INSERT INTO Students (Name, Age) VALUES
15     (N'Nguyễn Văn An', 21),
16     (N'Trần Thị Bình', 22),
17     (N'Lê Minh Châu', 20);
```

Listing 1: SchoolDB.sql

[x] Bước 2 — Tạo ASP.NET Web API Project

Mở Terminal / Command Prompt và chạy:

Listing 2: Terminal

```
# Tạo project mới
dotnet new webapi -n DapperApi
cd DapperApi

# Cài đặt NuGet packages
dotnet add package Dapper
dotnet add package Microsoft.Data.SqlClient
```

Cấu trúc thư mục sau khi tạo:

```
DapperApi/
|- Controllers/
|- Models/           <-- tạo thư mục này
|- Repositories/     <-- tạo thư mục này
|- appsettings.json
|- Program.cs
```

[x] Bước 3 — Tạo Model Class

Tạo thư mục Models/ và file Student.cs:

```
1 namespace DapperApi.Models;
2
3 public class Student
4 {
```

```
5     public int    Id    { get; set; }
6     public string Name { get; set; } = string.Empty;
7     public int    Age  { get; set; }
8 }
```

Listing 3: Models/Student.cs

[x] Bước 4 — Cấu hình Connection String

Mở appsettings.json và thêm:

```
1 {
2     "ConnectionStrings": {
3         "DefaultConnection":
4             "Server=localhost;Database=SchoolDB;Trusted_Connection=True;
5             TrustServerCertificate=True;"
6     },
7     "Logging": { ... }
8 }
```

Listing 4: appsettings.json

[!] Lưu ý

Nếu dùng **SQL Authentication** thay vì Windows Auth, connection string phải có dạng:

```
Server=localhost;Database=SchoolDB;User
Id=sa;Password=yourpassword;TrustServerCertificate=True;
```

[x] Bước 5 — Tạo Repository Interface

Tạo thư mục Repositories/ và file IStudentRepository.cs:

```
1 using DapperApi.Models;
2
3 namespace DapperApi.Repositories;
4
5 public interface IStudentRepository
6 {
7     IEnumerable<Student> GetAll();
8     Student?             GetById(int id);
9     void                 Create(Student student);
10    void                 Update(Student student);
11    void                 Delete(int id);
12 }
```

Listing 5: Repositories/IStudentRepository.cs

[x] Bước 6 — Implement Repository với Dapper

Tạo file StudentRepository.cs trong cùng thư mục Repositories/:

```
1 using Dapper;
2 using System.Data;
3 using Microsoft.Data.SqlClient;
4 using DapperApi.Models;
5
6 namespace DapperApi.Repositories;
7
8 public class StudentRepository : IStudentRepository
9 {
10     private readonly string _connStr;
11
12     public StudentRepository(IConfiguration config)
13     {
14         _connStr = config.GetConnectionString("DefaultConnection")!;
15     }
16
17     // Tao connection moi moi lan goi
18     private IDbConnection NewConnection()
19         => new SqlConnection(_connStr);
20
21     // GET ALL
22     public IEnumerable<Student> GetAll()
23     {
24         using var db = NewConnection();
25         return db.Query<Student>("SELECT * FROM Students");
26     }
27
28     // GET BY ID
29     public Student? GetById(int id)
30     {
31         using var db = NewConnection();
32         return db.QuerySingleOrDefault<Student>(
33             "SELECT * FROM Students WHERE Id = @Id",
34             new { Id = id });
35     }
36
37     // CREATE
38     public void Create(Student student)
39     {
40         using var db = NewConnection();
41         db.Execute(
42             "INSERT INTO Students (Name, Age) VALUES (@Name, @Age)",
43             student);
44     }
45
46     // UPDATE
47     public void Update(Student student)
48     {
49         using var db = NewConnection();
50         db.Execute(
51             "UPDATE Students SET Name=@Name, Age=@Age WHERE Id=@Id",
52             student);
53     }
```

```
54
55     // DELETE
56     public void Delete(int id)
57     {
58         using var db = NewConnection();
59         db.Execute(
60             "DELETE FROM Students WHERE Id=@Id",
61             new { Id = id });
62     }
63 }
```

Listing 6: Repositories/StudentRepository.cs

[x] Bước 7 — Đăng ký Dependency Injection

Mở Program.cs và thêm dòng sau **trước** builder.Build():

```
1 var builder = WebApplication.CreateBuilder(args);
2
3 // Đăng ký Repository vào DI container
4 builder.Services.AddScoped<IStudentRepository, StudentRepository>();
5
6 builder.Services.AddControllers();
7 builder.Services.AddEndpointsApiExplorer();
8 builder.Services.AddSwaggerGen();
9
10 var app = builder.Build();
11 // ...
```

Listing 7: Program.cs

[x] Bước 8 — Tạo Controller

Tạo file StudentsController.cs trong thư mục Controllers/:

```
1 using Microsoft.AspNetCore.Mvc;
2 using DapperApi.Models;
3 using DapperApi.Repositories;
4
5 namespace DapperApi.Controllers;
6
7 [ApiController]
8 [Route("api/[controller]")]
9 public class StudentsController : ControllerBase
10 {
11     private readonly IStudentRepository _repo;
12
13     public StudentsController(IStudentRepository repo)
14     {
15         _repo = repo;
16     }
17
18     // GET /api/students
19     [HttpGet]
20     public IActionResult GetAll()
```

```

21         => Ok(_repo.GetAll());
22
23     // GET /api/students/1
24     [HttpGet("{id:int}")]
25     public IActionResult Get(int id)
26     {
27         var student = _repo.GetById(id);
28         return student is null ? NotFound() : Ok(student);
29     }
30
31     // POST /api/students
32     [HttpPost]
33     public IActionResult Create([FromBody] Student student)
34     {
35         _repo.Create(student);
36         return CreatedAtAction(nameof(Get), new { id = student.Id },
37                                student);
38     }
39
40     // PUT /api/students
41     [HttpPut]
42     public IActionResult Update([FromBody] Student student)
43     {
44         _repo.Update(student);
45         return NoContent();
46     }
47
48     // DELETE /api/students/1
49     [HttpDelete("{id:int}")]
50     public IActionResult Delete(int id)
51     {
52         _repo.Delete(id);
53         return NoContent();
54     }

```

Listing 8: Controllers/StudentsController.cs

[x] Bước 9 — Chạy và kiểm tra API

Listing 9: Terminal

```
dotnet run
```

Mở trình duyệt tại <https://localhost:5001/swagger> để dùng Swagger UI.

Method	Endpoint	Mô tả	HTTP Status
GET	/api/students	Lấy tất cả sinh viên	200 OK
GET	/api/students/{id}	Lấy 1 sinh viên	200 / 404
POST	/api/students	Thêm sinh viên mới	201 Created
PUT	/api/students	Cập nhật sinh viên	204 No Content
DELETE	/api/students/{id}	Xoá sinh viên	204 No Content

5. Bài tập mở rộng (Advanced Exercises)

5.1. Bài tập 1 — Thêm trường Email

[*] Bài tập thực hành

1. Thêm cột Email NVARCHAR(150) vào bảng Students:

```
1 ALTER TABLE Students ADD Email NVARCHAR(150);
```

2. Cập nhật Student.cs để thêm property Email.
3. Sửa tất cả câu lệnh SQL trong StudentRepository.cs để xử lý trường Email.
4. Implement endpoint mới để tìm kiếm sinh viên theo tên:

```
GET /api/students/search?name=An
```

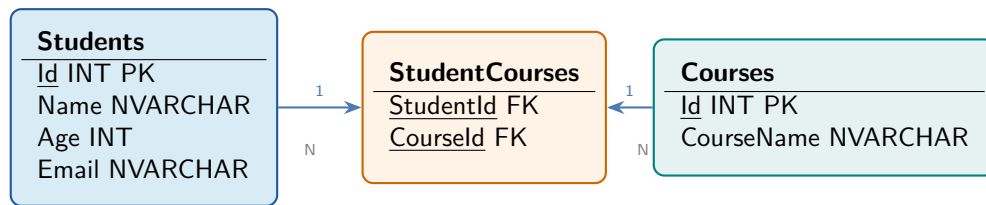
5.2. Bài tập 2 — Quan hệ nhiều-nhiều (Many-to-Many với JOIN)

Chuẩn bị Database

```
1 CREATE TABLE Courses
2 (
3     Id          INT          PRIMARY KEY IDENTITY,
4     CourseName NVARCHAR(100) NOT NULL
5 );
6
7 CREATE TABLE StudentCourses
8 (
9     StudentId INT NOT NULL,
10    CourseId   INT NOT NULL,
11    PRIMARY KEY (StudentId, CourseId),
12    FOREIGN KEY (StudentId) REFERENCES Students(Id),
13    FOREIGN KEY (CourseId)  REFERENCES Courses(Id)
14 );
15
16 -- Du lieu mau
17 INSERT INTO Courses (CourseName) VALUES
18     (N'ậLp trình .NET'), (N'ơC ỗ ữ ệliu'), (N'ếKin trúc ấphn ềmm');
19
20 INSERT INTO StudentCourses VALUES (1,1),(1,2),(2,2),(2,3),(3,1),(3,3);
```

Listing 10: Tạo bảng Courses và StudentCourses

Sơ đồ quan hệ (ERD)



Model cho kết quả JOIN

```

1 namespace DapperApi.Models;
2
3 public class Course
4 {
5     public int Id { get; set; }
6     public string CourseName { get; set; } = string.Empty;
7 }
8
9 public class StudentWithCourses
10 {
11     public int Id { get; set; }
12     public string Name { get; set; } = string.Empty;
13     public List<Course> Courses { get; set; } = new();
14 }
  
```

Listing 11: Models/StudentCourse.cs

Dapper Multi-Mapping

```

1 public IEnumerable<StudentWithCourses> GetAllWithCourses()
2 {
3     var sql = @"
4         SELECT s.Id, s.Name, c.Id, c.CourseName
5         FROM Students s
6         JOIN StudentCourses sc ON s.Id = sc.StudentId
7         JOIN Courses c ON sc.CourseId = c.Id
8         ORDER BY s.Id";
9
10    using var db = NewConnection();
11
12    var dict = new Dictionary<int, StudentWithCourses>();
13
14    db.Query<StudentWithCourses, Course, StudentWithCourses>(
15        sql,
16        (student, course) =>
17        {
18            if (!dict.TryGetValue(student.Id, out var existing))
19            {
20                existing = student;
21                dict[student.Id] = existing;
22            }
23            existing.Courses.Add(course);
24            return existing;
25        },
26        splitOn: "Id" // cot phan tach Student / Course
  
```

```
27     );  
28  
29     return dict.Values;  
30 }
```

Listing 12: Trong StudentRepository.cs

[*] Bài tập thực hành

1. Viết SQL JOIN query để lấy danh sách sinh viên kèm khóa học.
2. Implement method `GetAllWithCourses()` vào `IStudentRepository`.
3. Thêm endpoint mới vào Controller:

GET /api/students/courses

4. Test bằng Swagger và kiểm tra JSON response trả về.

6. Câu hỏi thảo luận

[?] Câu hỏi thảo luận

1. **Query vs Execute:** Sự khác nhau giữa `db.Query<T>()` và `db.Execute()` trong Dapper là gì? Khi nào dùng cái nào?
2. **Repository Pattern:** Tại sao cần tách logic database ra khỏi Controller? Lợi ích là gì khi test?
3. **Dapper vs EF Core:** So sánh ưu/nhược điểm của Dapper với Entity Framework Core. Khi nào nên chọn Dapper?
4. **Connection Management:** Tại sao mỗi phương thức trong Repository tạo `IDbConnection` mới thay vì dùng chung một connection?
5. **Dependency Injection:** Giải thích sự khác nhau giữa `AddScoped`, `AddTransient` và `AddSingleton`. Cái nào phù hợp cho Repository?